



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS220: Database Systems

Class: BSAI – 2A

Lab 13: Views in MYSQL

CLO-2: Formulate SQL queries to retrieve information from a relational database.

Instructor: Sara Tariq Sheikh

Lab Engineer: Saood Sarwar

Date: 30 April 2026

Time: 10:00 - 01:00



Introduction

This lab will focus on the types of database objects other than tables. In this lab you will practice defining, modifying and using views.

Objectives

After completing this lab, you should be able to do the following:

- Create a view
- Alter/Update a view
- Alter a table

Tools/Software Requirement

- MySQL Community Server 5.6
- MySQL Workbench 6.1

Helping Material

<https://www.mysqltutorial.org/mysql-view-with-check-option/>

Description

Views (including updatable views) are stored queries that when invoked produce a result set. A view acts as a virtual table.

CREATE VIEW/ ALTER VIEW

Syntax

The syntax for the CREATE VIEW statement in MySQL is:

```
CREATE [OR REPLACE] VIEW view_name AS  
SELECT columns  
FROM tables  
WHERE conditions  
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

OR REPLACE is optional. If you do not specify this clause and the VIEW already exists, the CREATE VIEW statement will return an error.



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

The CREATE VIEW statement creates a new view, or replaces an existing view if the OR REPLACE clause is given. If the view does not exist, CREATE OR REPLACE VIEW is the same as CREATE VIEW. If the view does exist, CREATE OR REPLACE VIEW is the same as ALTER VIEW.

The ALTER VIEW statement does the same thing as CREATE OR REPLACE.

The full ALTER statement looks like this:

```
ALTER [<algorithm attributes>] VIEW [<database>.]< name> [(<columns>)] AS  
<SELECT statement> [<check options>]
```

The select_statement is a SELECT statement that provides the definition of the view. (Selecting from the view selects, in effect, using the SELECT statement.) The select_statement can select from base tables or other views. The view definition is “frozen” at creation time. Changes to the underlying tables afterward do not affect the view definition. For example, if a view is defined as SELECT * on a table, new columns added to the table later do not become part of the view.

The CREATE VIEW statement requires the CREATE VIEW privilege for the view, and some privilege for each column selected by the SELECT statement. For columns used elsewhere in the SELECT statement, you must have the SELECT privilege. If the OR REPLACE clause is present, you must also have the DROP privilege for the view.



Updatable and Insertable Views

Some views are updatable and references to them can be used to specify tables to be updated in data change statements. That is, you can use them in statements such as UPDATE, DELETE, or INSERT to update the contents of the underlying table. Derived tables can also be specified in multiple-table UPDATE and DELETE statements, but can only be used for reading data to specify rows to be updated or deleted. Generally, the view references must be updatable, meaning that they may be merged and not materialized. Composite views have more complex rules.

For a view to be updatable there must be a one-to-one relationship between the rows in the view and the rows in the underlying table. There are also certain other constructs that make a view non-updatable. To be more specific, a view is not updatable if it contains any of the following:

- Aggregate functions (SUM(), MIN(), MAX(), COUNT(), and so forth)
- DISTINCT
- GROUP BY
- HAVING
- UNION or UNION ALL
- Subquery in the select list (fails for INSERT, okay for UPDATE, DELETE)
- Certain joins (see additional join discussion later in this section)
- Reference to non-updatable view in the FROM clause
- Subquery in the WHERE clause that refers to a table in the FROM clause
- Refers only to literal values (in this case, there is no underlying table to update)
- Multiple references to any column of a base table (fails for INSERT, okay for UPDATE, DELETE)

The View WITH CHECK OPTION Clause

The WITH CHECK OPTION clause can be given for an updatable view to prevent inserts to rows for which the WHERE clause in the select_statement is not true. It also prevents updates to rows for which the WHERE clause is true but the update would cause it to be not true (in other words, it prevents visible rows from being updated to nonvisible rows).



Lab Tasks

Given the following **database schema**:

Student (snum: integer, sname: char(30), major: char(25), level: char(2))

Faculty (fid: integer, fname: char(30), deptid: integer)

Class (cname: char(40), meets_at: char(20), room: char(10), fid: integer | fid REFS Faculty.fid)

Enrolled (snum: integer, cname: char(40) | snum REFS student.snum, cname REFS class.name)

Creating the tables:

```
CREATE TABLE Student (  
    snum INT PRIMARY KEY,  
    sname VARCHAR(30),  
    major VARCHAR(25),  
    level VARCHAR(2)  
);
```

```
CREATE TABLE Faculty (  
    fid INT PRIMARY KEY,  
    fname VARCHAR(30),  
    deptid INT  
);
```

```
CREATE TABLE Class (  
    cname VARCHAR(40) PRIMARY KEY,
```



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

meets_at VARCHAR(20),

room VARCHAR(10),

fid INT,

FOREIGN KEY (fid) REFERENCES Faculty(fid)

);

CREATE TABLE Enrolled (

snum INT,

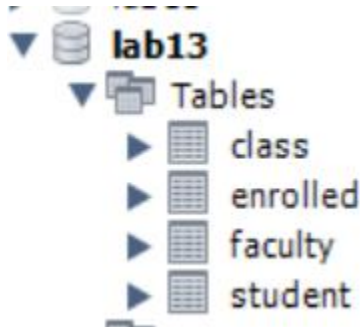
cname VARCHAR(40),

FOREIGN KEY (snum) REFERENCES Student(snum),

FOREIGN KEY (cname) REFERENCES Class(cname)

);

Proof:



Inserting values:

INSERT INTO Student VALUES

(1, 'Ali', 'Computer Science', 'JR'),

(2, 'Jack', 'Mathematics', 'SR'),



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

(3, 'Ahmed', 'Computer Science', 'JR'),

(4, 'Ebrahim', 'Physics', 'FR');

INSERT INTO Faculty VALUES

(10, 'Ivana Teach', 50),

(11, 'John Williams', 68),

(12, 'Christopher Garcia', 70),

(13, 'No Course Sir', 80);

INSERT INTO Class VALUES

('Database Systems', '9AM', 'R1', 10),

('Algorithms', '11AM', 'R2', 11),

('Physics 101', '1PM', 'R3', 12);

INSERT INTO Enrolled VALUES

(1, 'Database Systems'),

(2, 'Algorithms'),

(3, 'Database Systems'),

(4, 'Physics 101');



Proof:

Result Grid				Filter Rows:	
	cname	meets_at	room	fid	
▶	Algorithms	11AM	R2	11	
	Database Systems	9AM	R1	10	
	Physics 101	1PM	R3	12	
✱	NULL	NULL	NULL	NULL	

Class Table

Result Grid	Filter Rows:
snum	cname
1	Database Systems
2	Algorithms
3	Database Systems
4	Physics 101

Enrolled Table

Result Grid



Filter Rows:

	fid	fname	deptid
▶	10	Ivana Teach	50
	11	John Williams	68
	12	Christopher Garcia	70
	13	No Course Sir	80
⋮	NULL	NULL	NULL

Faculty Table



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

Result Grid				
Filter Rows:				
	snum	sname	major	level
▶	1	Ali	Computer Science	JR
	2	Jack	Mathematics	SR
	3	Ahmed	Computer Science	JR
	4	Ebrahim	Physics	FR
✱	NULL	NULL	NULL	NULL

Student Table

1. Create a view named v1 which has the name of faculty members who do not teach any course.

CODE:

```
CREATE VIEW v1 AS
```

```
SELECT fname
```

```
FROM Faculty
```

```
WHERE fid NOT IN (
```

```
    SELECT fid FROM Class
```

```
);
```

OUTPUT:

Result Grid	
	fname
▶	No Course Sir

2. Create another view named v2 which has the names of students who are enrolled in a course taught by faculty member "Ivana Teach".



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

CODE:

```
CREATE VIEW v2 AS
```

```
SELECT s.sname
```

```
FROM Student s
```

```
JOIN Enrolled e ON s.snum = e.snum
```

```
JOIN Class c ON e.cname = c.cname
```

```
JOIN Faculty f ON c.fid = f.fid
```

```
WHERE f.fname = 'Ivana Teach';
```

OUTPUT:

Result Grid		
	sname	
▶	Ali	
	Ahmed	

3. Create a view stdVu that is based on the student relation.

CODE:

```
CREATE VIEW stdVu AS
```

```
SELECT * FROM Student;
```

OUTPUT:

Result Grid





Filter Rows:

	snum	sname	major	level
▶	1	Ali	Computer Science	JR
	2	Jack	Mathematics	SR
	3	Ahmed	Computer Science	JR
	4	Ebrahim	Physics	FR



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

4. Alter the definition of student table. Add a column course to the student's relation.

CODE:

```
ALTER TABLE Student
```

```
ADD course VARCHAR(40);
```

OUTPUT:

Result Grid			Filter Rows:	Edit	
	snum	sname	major	level	course
▶	1	Ali	Computer Science	JR	NULL
	2	Jack	Mathematics	SR	NULL
	3	Ahmed	Computer Science	JR	NULL
	4	Ebrahim	Physics	FR	NULL
✱	NULL	NULL	NULL	NULL	NULL

5. Notice the changes in views which are based on student relation. Comment what happens to view data if the base table is modified.

COMMENT:

When the **Student** table is modified by adding a new column, the effect on views depends on how they are defined. If the same view is called which was made before altering table A (view stdvu) and then again after modifying the views original table which is the student table, as long as a column is added in the students table the view will not have any change. The same older view stdvu will be shown again.

Result Grid



Filter Rows:

snum	sname	major	level
1	Ali	Computer Science	JR
2	Jack	Mathematics	SR
3	Ahmed	Computer Science	JR
4	Ebrahim	Physics	FR

As we can see the same table is given again as in task 3



6. Alter view v2 based on the definition: It has the names of all juniors (Level = JR) who are enrolled in a class taught by 'Ivana Teach'.

CODE:

```
CREATE OR REPLACE VIEW v2 AS  
  
SELECT s.sname  
  
FROM Student s  
  
JOIN Enrolled e ON s.snum = e.snum  
  
JOIN Class c ON e.cname = c.cname  
  
JOIN Faculty f ON c.fid = f.fid  
  
WHERE f.fname = 'Ivana Teach'  
  
AND s.level = 'JR';
```

OUTPUT:

Result Grid	
	sname
▶	Ali
	Ahmed

7. Create views based on the following queries also:
a. The names of students majoring in 'Computer Science'.

CODE:



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
CREATE VIEW cs_students AS
```

```
SELECT sname
```

```
FROM Student
```

```
WHERE major = 'Computer Science';
```

OUTPUT:

Result Grid	
	sname
▶	Ali
	Ahmed

b. The names of classes taught by 'John Williams' in dept # 68.

CODE:

```
CREATE VIEW jw_classes AS
```

```
SELECT c.cname
```

```
FROM Class c
```

```
JOIN Faculty f ON c.fid = f.fid
```

```
WHERE f.fname = 'John Williams'
```

```
AND f.deptid = 68;
```

OUTPUT:

Result Grid	
	cname
▶	Algorithms

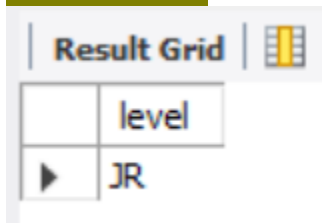
c. The distinct student ages in 'Database Systems' class in descending order.



CODE:

```
CREATE VIEW db_levels AS  
  
SELECT DISTINCT s.level  
  
FROM Student s  
  
JOIN Enrolled e ON s.snum = e.snum  
  
WHERE e.cname = 'Database Systems'  
  
ORDER BY s.level DESC;
```

OUTPUT:



	level
▶	JR

d. The name of 'Christopher Garcia's teachers.

CODE:

```
CREATE VIEW cg_students AS  
  
SELECT DISTINCT s.sname  
  
FROM Student s  
  
JOIN Enrolled e ON s.snum = e.snum  
  
JOIN Class c ON e.cname = c.cname  
  
JOIN Faculty f ON c.fid = f.fid  
  
WHERE f.fname = 'Christopher Garcia';
```

OUTPUT:



Result Grid	
	sname
▶	Ebrahim

8. Drop view v1, v2.

CODE:

```
DROP VIEW v1;
```

```
DROP VIEW v2;
```

OUTPUT:

Action Output			
	#	Time	Action
✓	40	11:11:32	DROP VIEW v1
✓	41	11:11:32	DROP VIEW v2

We can no longer access views v1 and v2 now

Deliverables

Complete your lab tasks in SQL workbench and submit a word file with queries and the screenshots of the results to all the questions attempted. Upload it on LMS. Late submissions will not be accepted.